id:1-1-1

Imports

In [24]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score
```

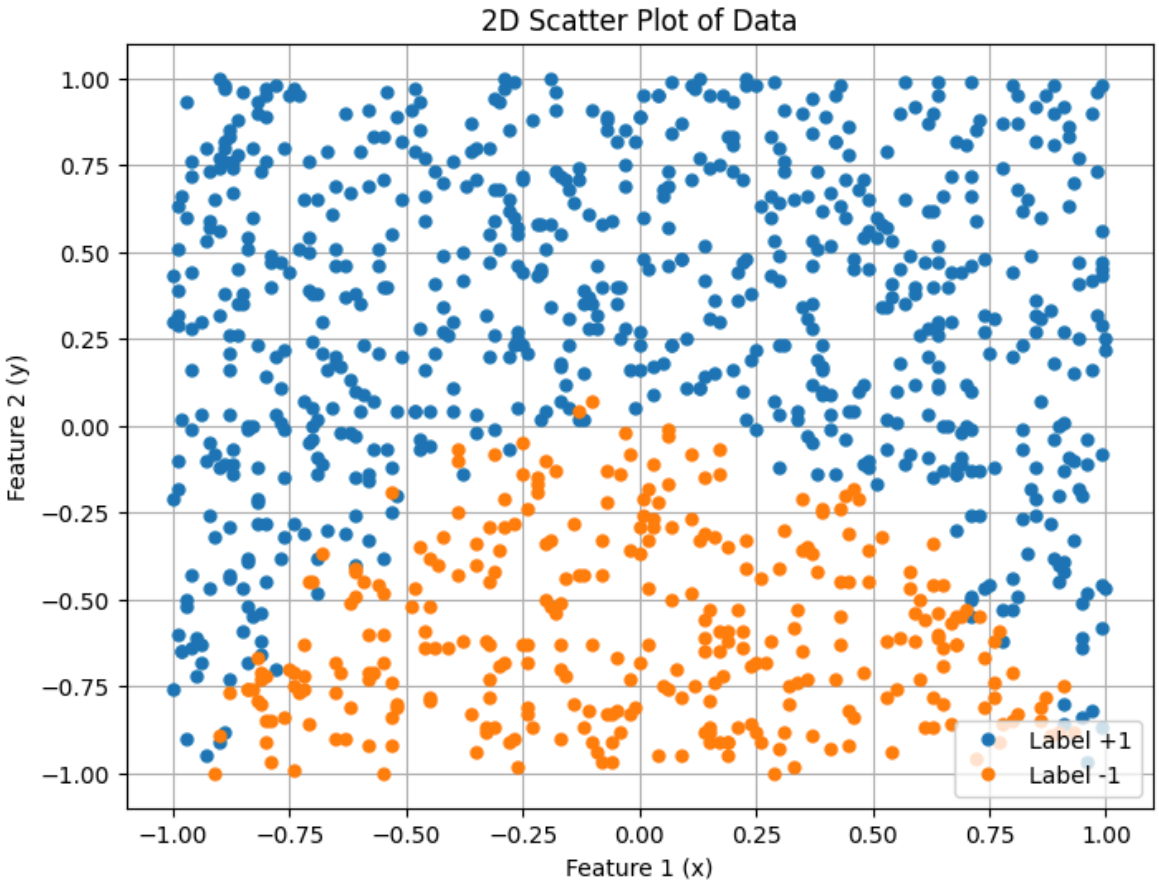
The data file consists of three columns of data (plus the first header line). The first two columns are input features and the third column is the real-valued target value.

In [25]:
```
df = pd.read_csv("data.csv")
print(df.head(10))
X1 = df.iloc[:, 0]
X2 = df.iloc[:, 1]
X = np.column_stack((X1, X2))
y = df.iloc[:, 2]
```

```
      x     y  label
0  0.64  0.30      1
1 -0.01  0.82      1
2 -0.82  0.23      1
3 -0.71  0.39      1
4 -0.47  0.28      1
5  0.49  0.45      1
6  0.41 -0.01      1
7  0.92  0.63      1
8 -0.10 -0.63     -1
9  0.05  0.66      1
```

2D Scatter Plot

In [26]:
```
plt.figure(figsize=(8, 6))


# Use boolean mask to filter rows X[y==n, :]
X_pos = X[y==1, :]
X_neg = X[y==-1, :]

plt.plot(X_pos[:, 0], X_pos[:, 1], '.', label='Label +1', markersize=10)
plt.plot(X_neg[:, 0], X_neg[:, 1], '.', label='Label -1', markersize=10)

plt.xlabel('Feature 1 (x)')
plt.ylabel('Feature 2 (y)')
plt.title('2D Scatter Plot of Data')
plt.legend()
plt.grid(True)
plt.show()
```



Logistic Regression

ii) Use sklearn to train a logistic regression classifier on the data. Give the logistic regression model for predictions and report the parameter values of the trained model. Discuss e.g. which feature has most influence on the prediction, which features cause the prediction to increase and which to decrease

In [27]:
```
# train the model
lr = LogisticRegression()
lr.fit(X, y)
```

Out[27]:

▾ LogisticRegression

▶ Parameters

In [28]:
```
lr.get_params()
```

Out[28]:
```
{'C': 1.0,
 'class_weight': None,
 'dual': False,
 'fit_intercept': True,
 'intercept_scaling': 1,
 'l1_ratio': None,
 'max_iter': 100,
 'multi_class': 'deprecated',
 'n_jobs': None,
 'penalty': 'l2',
 'random_state': None,
 'solver': 'lbfgs',
 'tol': 0.0001,
 'verbose': 0,
 'warm_start': False}
```

In [29]:
```
# Get the model parameters
print("Intercept:", lr.intercept_[0])
print("Feature 1 coef:", lr.coef_[0][0])
print("Feature 2 coef:", lr.coef_[0][1])
```

```
Intercept: 1.8001096150590803
Feature 1 coef: -0.2096025064278186
Feature 2 coef: 5.019054062423062
```

> iii) Now use the trained logistic regression classifier to predict the target values in the training data. Add these predictions to the 2D plot you generated in part (i), using a different marker and colour so that the training data and the predictions can be distinguished. Show the decision boundary of the logistic regression classifier as a line on the plot (you'll need to use the parameter values of the trained model to figure out what line this should be - explain how you obtain it).

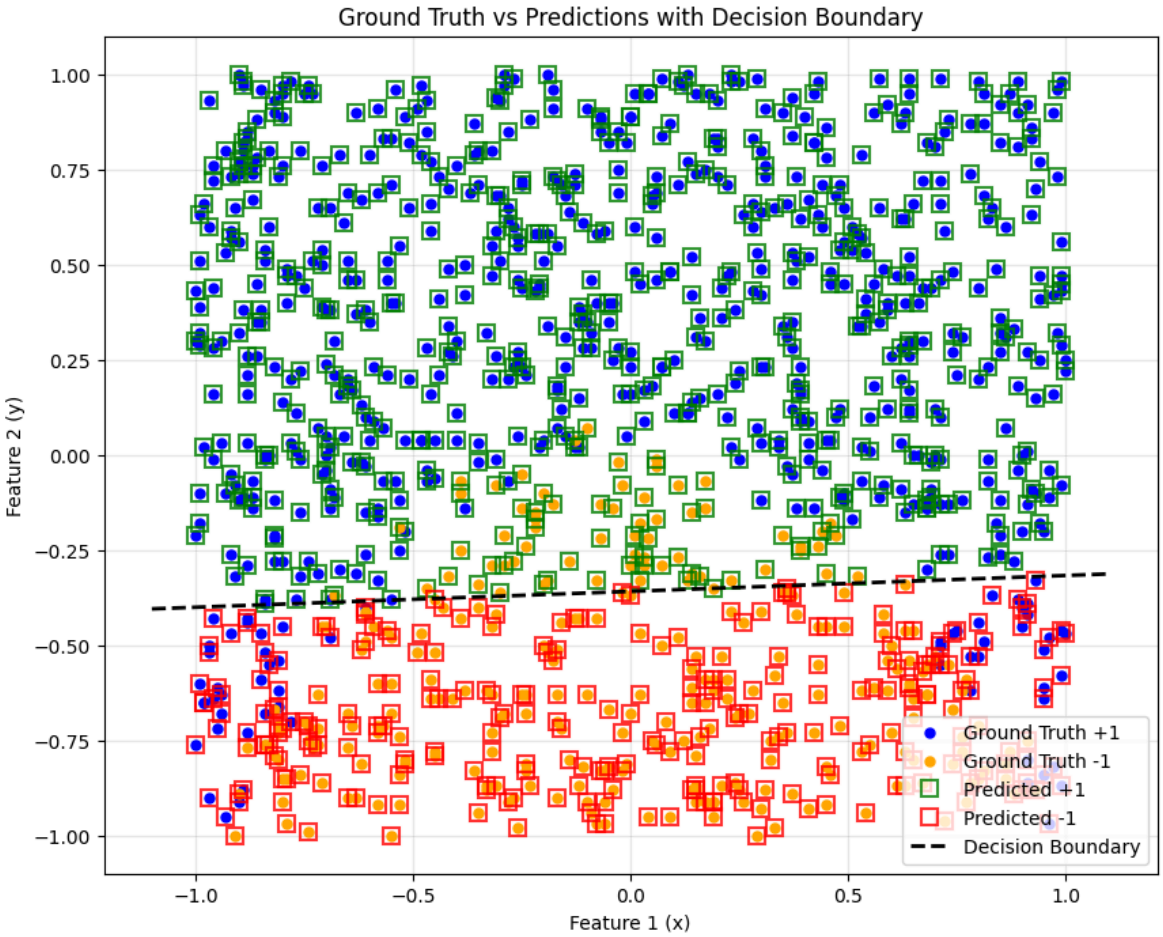
In [30]:
```
y_pred = lr.predict(X)
y_pred_proba = lr.predict_proba(X)

print("Predictions:", y_pred[:10])
print("Prediction probabilities:", y_pred_proba[:10])
```

```
Predictions: [ 1  1  1  1  1  1  1  1 -1  1]
Prediction probabilities: [[0.04024547 0.95975453]
 [0.00268381 0.99731619]
 [0.04203269 0.95796731]
 [0.01971703 0.98028297]
 [0.03543566 0.96456434]
 [0.01878047 0.98121953]
 [0.15922818 0.84077182]
 [0.00841508 0.99158492]
 [0.79264353 0.20735647]
 [0.00604653 0.99395347]]
```

In [31]:
```
X_pos_pred = X[y_pred == 1, :]
X_neg_pred = X[y_pred == -1, :]
```

In [67]:

```
plt.figure(figsize=(10, 8))

X_pos = X[y==1, :]
X_neg = X[y==-1, :]

# plot GT
plt.plot(X_pos[:, 0], X_pos[:, 1], '.', label='Ground Truth +1', markersi
plt.plot(X_neg[:, 0], X_neg[:, 1], '.', label='Ground Truth -1', markersi

# plot predictions
plt.plot(X_pos_pred[:, 0], X_pos_pred[:, 1], 's', label='Predicted +1', m
            color='green', alpha=0.8, fillstyle='none', markeredgewidth=1

plt.plot(X_neg_pred[:, 0], X_neg_pred[:, 1], 's', label='Predicted -1', m
            color='red', alpha=0.8, fillstyle='none', markeredgewidth=1.5

# decision boundary
# Step 1: Extract model parameters
intercept = lr.intercept_[0]  # bias term
coef = lr.coef_[0]           # [coef_x1, coef_x2]

# Step 2: Decision boundary occurs when P(y=1|x) = 0.5
# happens when z = 0
# Step 3: Solve for x2 in terms of x1

# Step 4: Create range of x1 values
x1_min = X[:, 0].min() - 0.1
x1_max = X[:, 0].max() + 0.1
x1_range = np.linspace(x1_min, x1_max, 100)

# Step 5: Calculate corresponding x2 values for decision boundary
x2_boundary = -(intercept + coef[0] * x1_range) / coef[1]


# Step 6: Plot decision boundary
plt.plot(x1_range, x2_boundary, 'k--', linewidth=2, label='Decision Bound

plt.xlabel('Feature 1 (x)')
plt.ylabel('Feature 2 (y)')
plt.title('Ground Truth vs Predictions with Decision Boundary')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



iv) Briefly comment on how the predictions and the training data compare.

get some data to back up

In [10]:
```
# Calculate additional metrics
accuracy = accuracy_score(y, y_pred)
precision = precision_score(y, y_pred)
recall = recall_score(y, y_pred)
f1 = f1_score(y, y_pred)

print(f"\n=== Performance Metrics ===")
print(f"Accuracy:  {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall:    {recall:.4f}")
print(f"F1-Score:  {f1:.4f}")
```

```
=== Performance Metrics ===
Accuracy:  0.8719
Precision: 0.9045
Recall:    0.9098
F1-Score:  0.9071
```

i) Train linear SVM classifiers for a wide range of values of the penalty parameter C e.g. C = 0.001, C = 1, C = 100. Give the SVM model for predictions and report

the parameter values of each trained model.

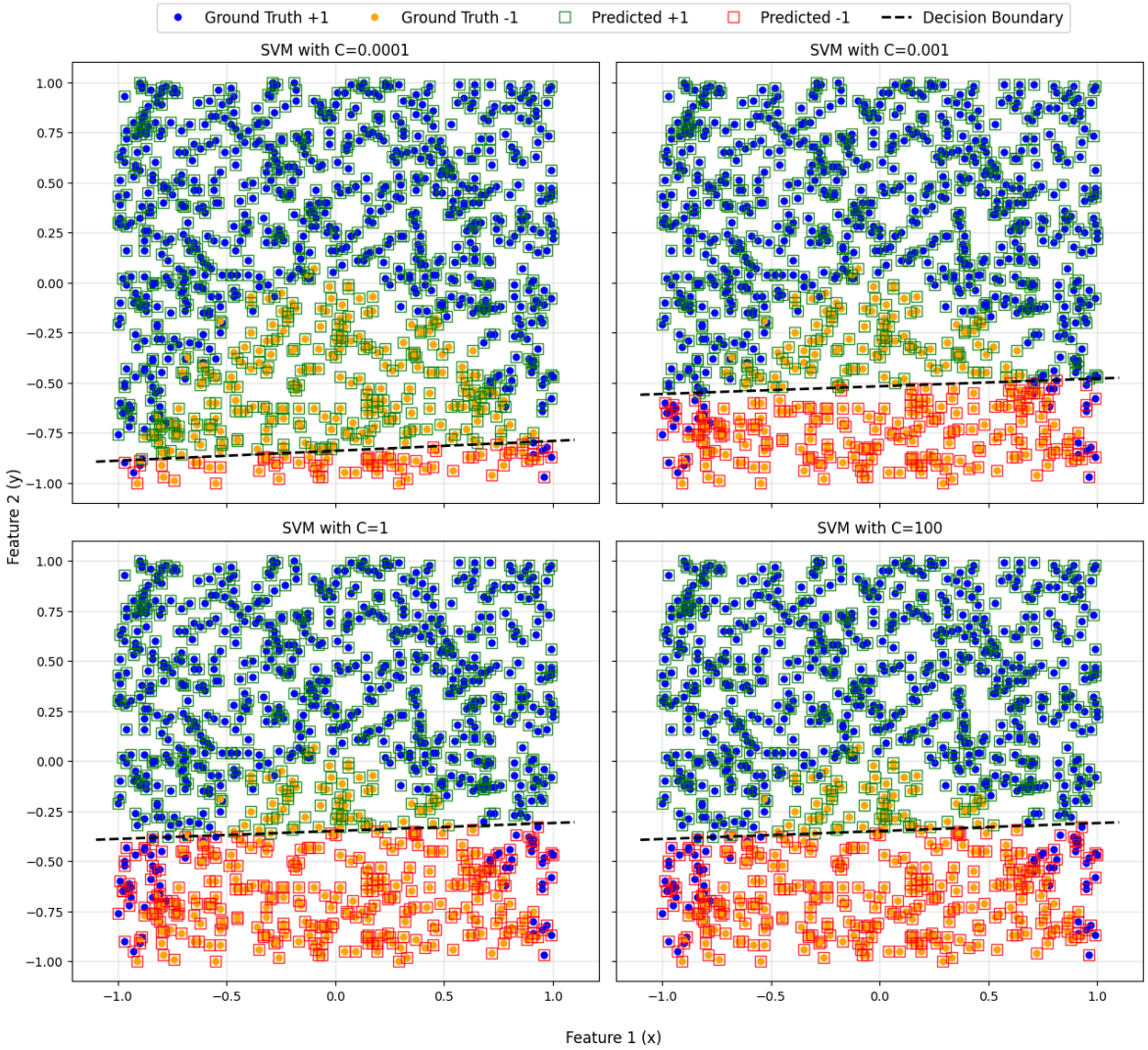
In [11]:
```
from sklearn.svm import LinearSVC

# Train linear SVM classifiers for a wide range of C values
C_values = [0.0001, 0.001, 1, 100]
svm_models = {}
for C in C_values:
    svm = LinearSVC(C=C, max_iter=10000, dual=False)
    svm.fit(X, y)
    svm_models[C] = svm
    print(f"\nSVM model for C={C}:")
    print("  Intercept:", svm.intercept_)
    print("  Coefficients:", svm.coef_)

    y_pred = svm.predict(X)
```

```
SVM model for C=0.0001:
  Intercept: [0.06219854]
  Coefficients: [[-0.00364037  0.07394777]]

SVM model for C=0.001:
  Intercept: [0.24237754]
  Coefficients: [[-0.01796787  0.46712054]]

SVM model for C=1:
  Intercept: [0.6448363]
  Coefficients: [[-0.07336485  1.84353925]]

SVM model for C=100:
  Intercept: [0.65218586]
  Coefficients: [[-0.07433828  1.86233769]]
```

ii) Use each of these trained classifiers to predict the target values in the training data. Plot these predictions and the actual target values from the data, together with the classifier decision boundary.

In [71]:
```
fig, axes = plt.subplots(2, 2, figsize=(14, 12), sharex=True, sharey=True
axes = axes.flatten()  # Flatten to make indexing easier

for idx, C in enumerate(C_values):
    ax = axes[idx]
    svm = svm_models[C]

    y_pred = svm.predict(X)

    # gt
    X_pos = X[y==1, :]
    X_neg = X[y==-1, :]

    # predictions
    X_pos_pred = X[y_pred==1, :]
    X_neg_pred = X[y_pred==-1, :]
```

```
    # plot gt
    ax.plot(X_pos[:, 0], X_pos[:, 1], '.', label='Ground Truth +1', marke
    ax.plot(X_neg[:, 0], X_neg[:, 1], '.', label='Ground Truth -1', marke

    # plot pred
    ax.plot(X_pos_pred[:, 0], X_pos_pred[:, 1], 's', label='Predicted +1'
            color='green', alpha=0.8, fillstyle='none', markeredgewidth=1

    ax.plot(X_neg_pred[:, 0], X_neg_pred[:, 1], 's', label='Predicted -1'
            color='red', alpha=0.8, fillstyle='none', markeredgewidth=1)

    # decision boundary
    intercept = svm.intercept_[0]
    coef = svm.coef_[0]

    # create a range of x1 values for the decision boundary
    x1_range = np.linspace(X[:, 0].min() - 0.1, X[:, 0].max() + 0.1, 100)
    # calculate corresponding x2 values for the decision boundary
    x2_boundary = -(intercept + coef[0] * x1_range) / coef[1]

    # plot
    ax.plot(x1_range, x2_boundary, 'k--', linewidth=2, label='Decision Bo

    ax.set_title(f'SVM with C={C}')
    ax.grid(True, alpha=0.3)

# shared axis
fig.supxlabel('Feature 1 (x)', fontsize=12)
fig.supylabel('Feature 2 (y)', fontsize=12)

# shared legend
handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, loc='upper center', bbox_to_anchor=(0.5, 0.98

plt.tight_layout(rect=[0, 0.01, 0.93, 0.95])
plt.show()
```



In [ ]:
```
print("="*100)
print("SVM MODEL PARAMETERS COMPARISON TABLE")
print("="*100)

print(f"{'C':<12} {'w1 (coef[0])':>15} {'w2 (coef[1])':>15} {'w0 (interce
print("="*100)

# Extract parameters for each model
for C in C_values:
    svm = svm_models[C]

    # Get model parameters
    coef = svm.coef_[0]
    intercept = svm.intercept_[0]
    w_norm = np.linalg.norm(coef)
    margin = 2 / w_norm

    # Print row
    print(f"{C:<12.4f} {coef[0]:>15.6f} {coef[1]:>15.6f} {intercept:>15.6

print("="*100)
```

```
================================================================================
============================
SVM MODEL PARAMETERS COMPARISON TABLE
================================================================================
============================
C               w1 (coef[0])    w2 (coef[1])   b (intercept)        ||w||
Margin (2/||w||)
================================================================================
============================
0.0001            -0.003640        0.073948        0.062199     0.074037
27.013404
0.0010            -0.017968        0.467121        0.242378     0.467466
4.278386
1.0000            -0.073365        1.843539        0.644836     1.844998
1.084012
100.0000          -0.074338        1.862338        0.652186     1.863821
1.073065
================================================================================
============================
```

In [17]:

```python
# performance metrics for each SVM model
print("="*80)
print("SVM PERFORMANCE METRICS FOR DIFFERENT C VALUES")
print("="*80)

# Store metrics for comparison
metrics_table = []

for C in C_values:
    svm = svm_models[C]
    y_pred = svm.predict(X)

    # cacl metrics
    accuracy = accuracy_score(y, y_pred)
    precision = precision_score(y, y_pred)
    recall = recall_score(y, y_pred)
    f1 = f1_score(y, y_pred)

    # confusyioin matrix
    cm = confusion_matrix(y, y_pred)
    TN, FP, FN, TP = cm.ravel()

    TPR = TP / (TP + FN)
    FPR = FP / (FP + TN)
    TNR = TN / (TN + FP)
    FNR = FN / (FN + TP)

    print(f"\n{'='*80}")
    print(f"SVM with C = {C}")
    print(f"{'='*80}")
    print(f"Accuracy:  {accuracy:.4f} ({accuracy*100:.2f}%)")
    print(f"Precision: {precision:.4f}")
    print(f"Recall:    {recall:.4f}")
    print(f"F1-Score:  {f1:.4f}")
    print(f"\nConfusion Matrix:")
    print(f"  TN: {TN:3d}  FP: {FP:3d}")
    print(f"  FN: {FN:3d}  TP: {TP:3d}")
    print(f"\nClassification Rates:")
    print(f"  TPR (Sensitivity): {TPR:.4f}")
    print(f"  TNR (Specificity): {TNR:.4f}")
```

```
    print(f"  FPR: {FPR:.4f}")
    print(f"  FNR: {FNR:.4f}")

    metrics_table.append({
        'C': C,
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall,
        'F1-Score': f1,
        'TPR': TPR,
        'TNR': TNR
    })

# print comparison table
print(f"\n{'='*80}")
print(f"COMPARISON TABLE")
print(f"{'='*80}")
print(f"{'C':<12} {'Accuracy':>10} {'Precision':>10} {'Recall':>10} {'F1-
print(f"{'-'*80}")

for metrics in metrics_table:
    print(f"{metrics['C']:<12.4f} {metrics['Accuracy']:>10.4f} {metrics['
          f"{metrics['Recall']:>10.4f} {metrics['F1-Score']:>10.4f} {metr

print(f"{'='*80}")
```

```
================================================================================
======
SVM PERFORMANCE METRICS FOR DIFFERENT C VALUES
================================================================================
======

================================================================================
======
SVM with C = 0.0001
================================================================================
======
Accuracy:  0.7427 (74.27%)
Precision: 0.7322
Recall:    0.9869
F1-Score:  0.8407

Confusion Matrix:
  TN:  64  FP: 248
  FN:   9  TP: 678

Classification Rates:
  TPR (Sensitivity): 0.9869
  TNR (Specificity): 0.2051
  FPR: 0.7949
  FNR: 0.0131

================================================================================
======
SVM with C = 0.001
================================================================================
======
Accuracy:  0.8519 (85.19%)
Precision: 0.8523
Recall:    0.9491
F1-Score:  0.8981

Confusion Matrix:
  TN: 199  FP: 113
  FN:  35  TP: 652

Classification Rates:
  TPR (Sensitivity): 0.9491
  TNR (Specificity): 0.6378
  FPR: 0.3622
  FNR: 0.0509

================================================================================
======
SVM with C = 1
================================================================================
======
Accuracy:  0.8699 (86.99%)
Precision: 0.9054
Recall:    0.9054
F1-Score:  0.9054

Confusion Matrix:
  TN: 247  FP:  65
  FN:  65  TP: 622
```

```
Classification Rates:
  TPR (Sensitivity): 0.9054
  TNR (Specificity): 0.7917
  FPR: 0.2083
  FNR: 0.0946

================================================================================
======
SVM with C = 100
================================================================================
======
Accuracy:  0.8699 (86.99%)
Precision: 0.9054
Recall:    0.9054
F1-Score:  0.9054

Confusion Matrix:
  TN: 247  FP:  65
  FN:  65  TP: 622

Classification Rates:
  TPR (Sensitivity): 0.9054
  TNR (Specificity): 0.7917
  FPR: 0.2083
  FNR: 0.0946

================================================================================
======
COMPARISON TABLE
================================================================================
======
C              Accuracy  Precision     Recall   F1-Score        TPR
TNR
--------------------------------------------------------------------------------
------
0.0001           0.7427     0.7322     0.9869     0.8407     0.9869     0.
2051
0.0010           0.8519     0.8523     0.9491     0.8981     0.9491     0.
6378
1.0000           0.8699     0.9054     0.9054     0.9054     0.9054     0.
7917
100.0000         0.8699     0.9054     0.9054     0.9054     0.9054     0.
7917
================================================================================
======
```

> iii) What is the impact on the model parameters of changing C, and why ? What is the impact on the SVM predictions?

Practical Implications C too small: Underfitting - model is too simple C too large: Overfitting - model memorizes training data Optimal C: Found through cross-validation, balances bias and variance

> iv) How do the SVM model parameters and predictions compare to those of the logistic regression model in part (a)?

Part C

> i) Now create two additional features by adding the square of each feature (i.e. giving four features in total). Train a logistic regression classifier. Give the model and the trained parameter values.

In [18]:
```
# polynomial features (original + squared)
X_poly = np.column_stack((X1, X2, X1**2, X2**2))

print("Original features shape:", X.shape)
print("Polynomial features shape:", X_poly.shape)
print("Feature names: [X1, X2, X1², X2²]")
print(X_poly[:2])
```

```
Original features shape: (999, 2)
Polynomial features shape: (999, 4)
Feature names: [X1, X2, X1², X2²]
[[ 6.400e-01  3.000e-01  4.096e-01  9.000e-02]
 [-1.000e-02  8.200e-01  1.000e-04  6.724e-01]]
```

In [19]:
```
clf_poly = LogisticRegression()
clf_poly.fit(X_poly, y)
```

Out[19]:

▾ LogisticRegression

▶ Parameters

In [20]:
```
print("=== Logistic Regression with Polynomial Features ===")
print("Intercept (bias):", clf_poly.intercept_[0])
print("Coefficients:", clf_poly.coef_[0])
print("\nFeature coefficients:")
print(f"X1 (x) coefficient: {clf_poly.coef_[0][0]:.4f}")
print(f"X2 (y) coefficient: {clf_poly.coef_[0][1]:.4f}")
print(f"X1² (x²) coefficient: {clf_poly.coef_[0][2]:.4f}")
print(f"X2² (y²) coefficient: {clf_poly.coef_[0][3]:.4f}")
```

```
=== Logistic Regression with Polynomial Features ===
Intercept (bias): 0.2984504002925016
Coefficients: [-0.15002377  6.96714254  6.44457421 -0.54193648]

Feature coefficients:
X1 (x) coefficient: -0.1500
X2 (y) coefficient: 6.9671
X1² (x²) coefficient: 6.4446
X2² (y²) coefficient: -0.5419
```

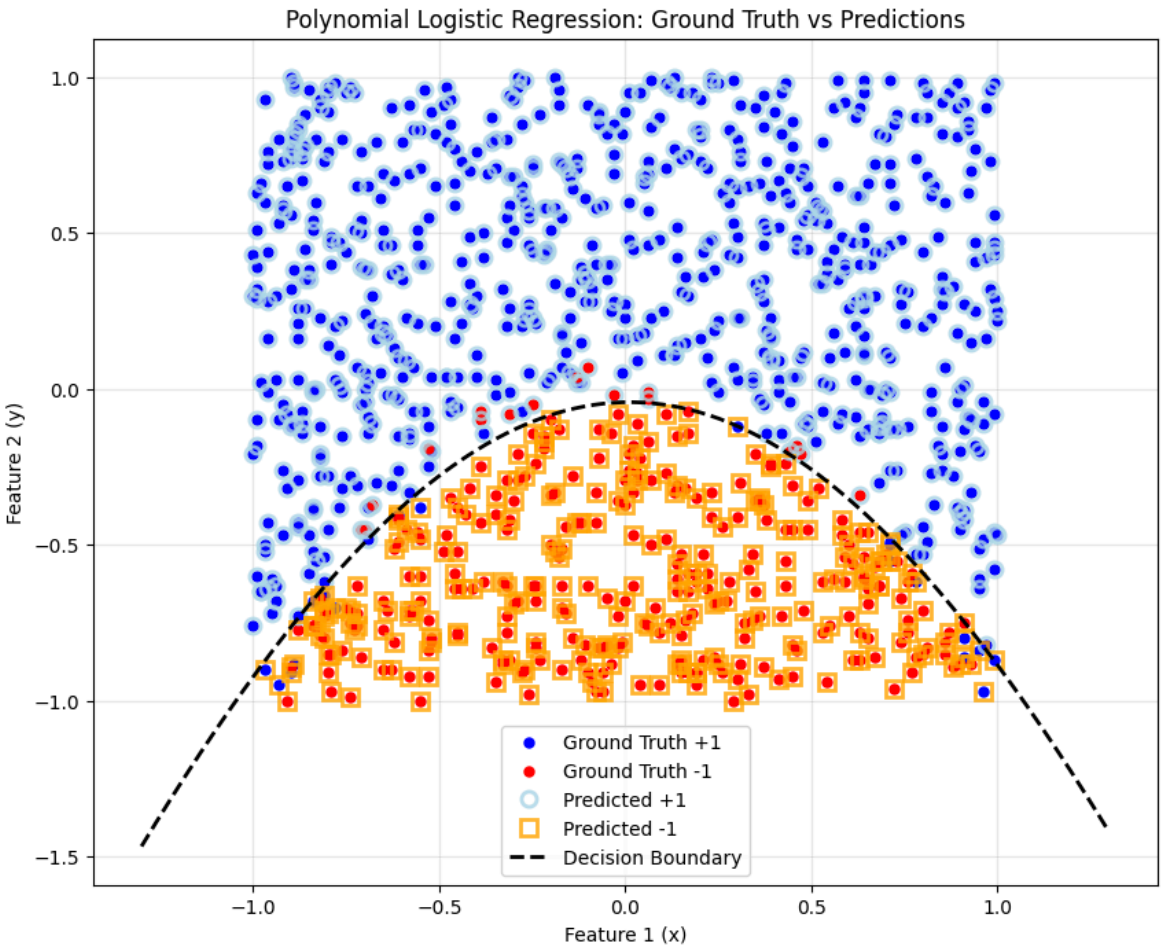
> ii) Use the trained classifier to predict the target values in the training data. Plot these predictions and the actual target values from the data using the same style of plot as before i.e. using just the two original features as x and y axes. Compare and discuss. How do the predictions compare with those in parts (a) and (b) above?

In [22]:

```
y_pred_poly = clf_poly.predict(X_poly)
y_pred_proba_poly = clf_poly.predict_proba(X_poly)

print("Polynomial LR Predictions:", y_pred_poly[:10])
print("Polynomial LR Prediction probabilities:", y_pred_proba_poly[:10])

plt.figure(figsize=(10, 8))

X_pos = X[y==1, :]
X_neg = X[y==-1, :]

X_pos_pred = X[y_pred_poly==1, :]
X_neg_pred = X[y_pred_poly==-1, :]

# ground truth
plt.plot(X_pos[:, 0], X_pos[:, 1], '.', label='Ground Truth +1', markersi
plt.plot(X_neg[:, 0], X_neg[:, 1], '.', label='Ground Truth -1', markersi

# predictions
plt.plot(X_pos_pred[:, 0], X_pos_pred[:, 1], 'o', label='Predicted +1', m
            color='lightblue', alpha=0.8, fillstyle='none', markeredgewid

plt.plot(X_neg_pred[:, 0], X_neg_pred[:, 1], 's', label='Predicted -1', m
            color='orange', alpha=0.8, fillstyle='none', markeredgewidth=

intercept = clf_poly.intercept_[0]
coef = clf_poly.coef_[0]

print(f"Decision boundary equation:")
print(f"{coef[3]:.4f}*X2² + {coef[1]:.4f}*X2 + ({intercept:.4f} + {coef[0

# Create range of X1 values
x1_range = np.linspace(X[:, 0].min() - 0.3, X[:, 0].max() + 0.3, 200)

# For each X1, solve quadratic equation for X2
a = coef[3]
b = coef[1]
c_values = intercept + coef[0]*x1_range + coef[2]*x1_range**2
```

```python
# Quadratic formula: X2 = (-b +- sqrt(b2 - 4ac)) / (2a)
discriminant = b**2 - 4*a*c_values

# Only plot where discriminant >= 0 (real solutions)
valid_idx = discriminant >= 0
x1_valid = x1_range[valid_idx]
disc_valid = discriminant[valid_idx]

# Two solutions (+ and -)
x2_plus = (-b + np.sqrt(disc_valid)) / (2*a)
x2_minus = (-b - np.sqrt(disc_valid)) / (2*a)

# Plot both branches of the boundary
plt.plot(x1_valid, x2_plus, 'k--', linewidth=2, label='Decision Boundary'

# Wrong plot
# plt.plot(x1_valid, x2_minus, 'k--', linewidth=2)

plt.xlabel('Feature 1 (x)')
plt.ylabel('Feature 2 (y)')
plt.title('Polynomial Logistic Regression: Ground Truth vs Predictions')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

```
Polynomial LR Predictions: [ 1  1  1  1  1  1  1  1 -1  1]
Polynomial LR Prediction probabilities: [[7.51358011e-03 9.92486420e-01]
 [3.50779427e-03 9.96492206e-01]
 [1.78139428e-03 9.98218606e-01]
 [1.85422190e-03 9.98145778e-01]
 [2.41063081e-02 9.75893692e-01]
 [8.18078262e-03 9.91819217e-01]
 [2.22605298e-01 7.77394702e-01]
 [5.60437790e-05 9.99943956e-01]
 [9.85606718e-01 1.43932816e-02]
 [9.29162101e-03 9.90708379e-01]]
Decision boundary equation:
-0.5419*X2² + 6.9671*X2 + (0.2985 + -0.1500*X1 + 6.4446*X1²) = 0
```



In [23]:
```python
from sklearn.metrics import accuracy_score, precision_score, recall_score

# baseline
unique_classes, counts = np.unique(y, return_counts=True)
most_common_class = unique_classes[np.argmax(counts)]

print(f"Most common class in dataset: {most_common_class}")
print(f"Class distribution:")
for cls, count in zip(unique_classes, counts):
    print(f"  Class {cls:>2}: {count} samples ({count/len(y)*100:.1f}%)")

y_pred_baseline = np.full_like(y, most_common_class)


# predict
y_pred_poly = clf_poly.predict(X_poly)

# from previous
def calculate_metrics(y_true, y_pred, model_name):
    """Calculate and return all performance metrics"""

    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred)
    recall = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)

    # Confusion matrix
    # Possibly not needed but was used in biometrics course
    cm = confusion_matrix(y_true, y_pred)
    TN, FP, FN, TP = cm.ravel()
```

```
    # Rates
    TPR = TP / (TP + FN)
    FPR = FP / (FP + TN)
    TNR = TN / (TN + FP)
    FNR = FN / (FN + TP)

    print(f"\n{'='*50}")
    print(f"{model_name}")
    print(f"{'='*50}")
    print(f"Accuracy:  {accuracy:.4f} ({accuracy*100:.2f}%)")
    print(f"Precision: {precision:.4f}")
    print(f"Recall:    {recall:.4f}")
    print(f"F1-Score:  {f1:.4f}")
    print(f"\nConfusion Matrix:")
    print(f"  TN: {TN:3d}  FP: {FP:3d}")
    print(f"  FN: {FN:3d}  TP: {TP:3d}")
    print(f"\nRates:")
    print(f"  TPR (Sensitivity): {TPR:.4f}")
    print(f"  TNR (Specificity): {TNR:.4f}")
    print(f"  FPR: {FPR:.4f}")
    print(f"  FNR: {FNR:.4f}")

    return {
        'accuracy': accuracy,
        'precision': precision,
        'recall': recall,
        'f1': f1,
        'TPR': TPR,
        'TNR': TNR,
        'FPR': FPR,
        'FNR': FNR
    }

baseline_metrics = calculate_metrics(y, y_pred_baseline, "BASELINE (Alway
poly_lr_metrics = calculate_metrics(y, y_pred_poly, "POLYNOMIAL LOGISTIC


print(f"\n{'='*70}")
print(f"PERFORMANCE COMPARISON")
print(f"{'='*70}")
print(f"{'Metric':<20} {'Baseline':>15} {'Poly LR':>15} {'Improvement':>1
print(f"{'-'*70}")

for metric_name in ['accuracy', 'precision', 'recall', 'f1']:
    baseline_val = baseline_metrics[metric_name]
    poly_val = poly_lr_metrics[metric_name]
    improvement = poly_val - baseline_val

    print(f"{metric_name.capitalize():<20} {baseline_val:>15.4f} {poly_va

print(f"{'='*70}")
```

```
Most common class in dataset: 1
Class distribution:
  Class -1: 312 samples (31.2%)
  Class  1: 687 samples (68.8%)

==================================================
BASELINE (Always Predict Most Common Class)
==================================================
Accuracy:  0.6877 (68.77%)
Precision: 0.6877
Recall:    1.0000
F1-Score:  0.8149

Confusion Matrix:
  TN:   0  FP: 312
  FN:   0  TP: 687

Rates:
  TPR (Sensitivity): 1.0000
  TNR (Specificity): 0.0000
  FPR: 1.0000
  FNR: 0.0000

==================================================
POLYNOMIAL LOGISTIC REGRESSION
==================================================
Accuracy:  0.9640 (96.40%)
Precision: 0.9752
Recall:    0.9723
F1-Score:  0.9738

Confusion Matrix:
  TN: 295  FP:  17
  FN:  19  TP: 668

Rates:
  TPR (Sensitivity): 0.9723
  TNR (Specificity): 0.9455
  FPR: 0.0545
  FNR: 0.0277

======================================================================
PERFORMANCE COMPARISON
======================================================================
Metric                      Baseline         Poly LR     Improvement
----------------------------------------------------------------------
Accuracy                      0.6877          0.9640         +0.2763
Precision                     0.6877          0.9752         +0.2875
Recall                        1.0000          0.9723         -0.0277
F1                            0.8149          0.9738         +0.1588
======================================================================
```